The Vector Database, Explained

Every concept in the vecdb project: what it is, how this project uses it, why it was chosen, where the industry uses it, and how it makes production systems faster.

Python · FAISS · IVF-PQ · HNSW · SQLite · gRPC · Protobuf · Sharding · RAG · Gemini

Project P0 · October 2026 · numbers measured on Apple M1, 8 GB RAM

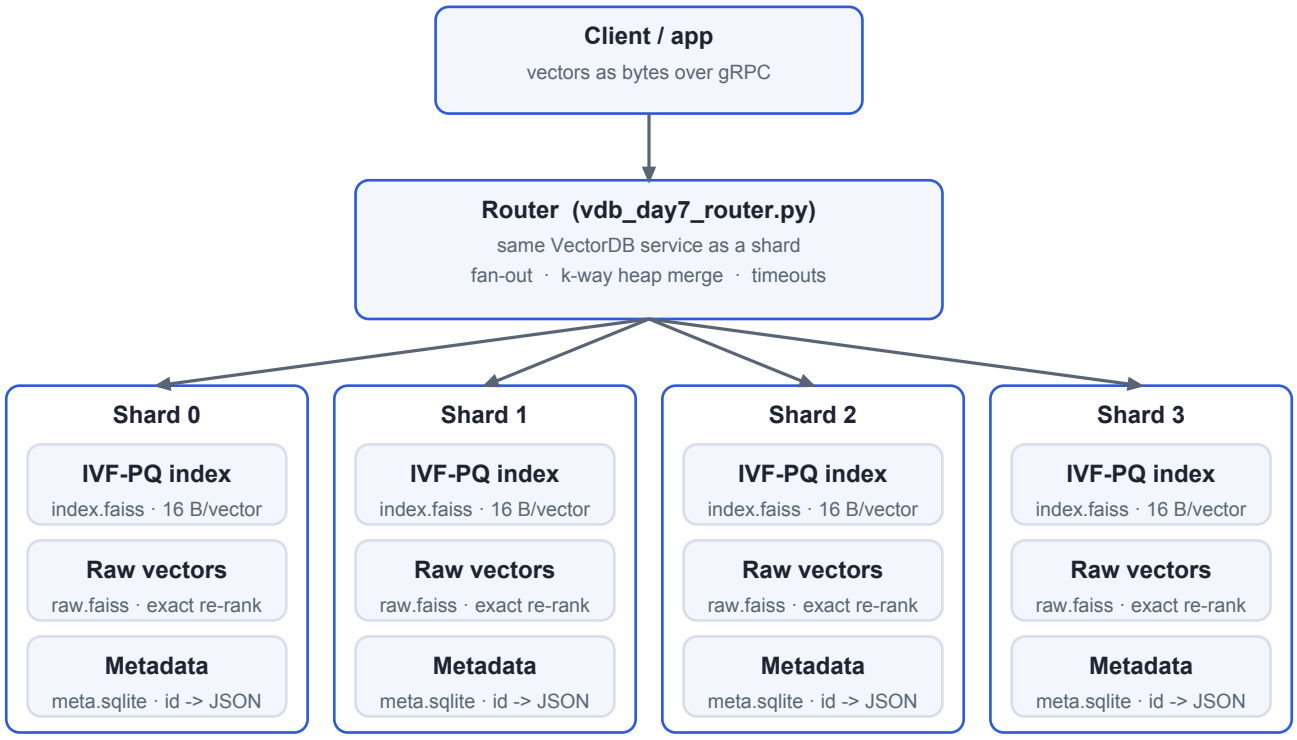
Contents

1. What you built, in one page 3
2. System design: what it is and how this project does it 5
3. Embeddings and vectors 7
4. Distance metrics: L2 and inner product 8
5. Exact search, ground truth, and recall@k 9
6. Approximate nearest neighbour (ANN) and LSH 10
7. HNSW: the graph index 11
8. IVF: the inverted file (partitioning) 12
9. PQ: product quantization (compression) 13
10. IVF-PQ + exact re-ranking 14
11. FAISS: the engine 16
12. Benchmarking like a database engineer 17
13. The database layer (VectorDB class) 18
14. RAG: retrieval-augmented generation 20
15. Protobuf and gRPC 21
16. Sharding, scatter-gather and the heap merge 22
17. Building at scale: streaming, sampling, checkpointing 25
18. The 1B run: cloud, memory math and cost 26
19. Testing: proving it works 27
20. Summary: every technique and what it buys 28
21. Numbers and resume lines 29
22. Glossary 30

1. What you built, in one page

A vector database stores millions of vectors (lists of numbers that represent meaning) and, given a new vector, returns the closest ones fast. You built the full stack around that idea:

Layer	File	What it does
Index engine	FAISS (library)	IVF, PQ, HNSW, Flat: the nearest-neighbour algorithms, compiled C++.
Database	`vdb_day5_db.py`	`VectorDB` class: ids, metadata (SQLite), save/load, mmap, re-rank, thread safety.
Application	`vdb_day5_rag.py`	RAG: embed facts once, retrieve with a calibrated floor, answer with Gemini.
Wire format	`vecdb.proto`	Protobuf messages + the gRPC service both shard and router speak.
Shard server	`vdb_day6_shard.py`	One process serves one VectorDB directory over gRPC.
Router	`vdb_day7_router.py`	Fans a query out to N shards in parallel, merges top-k with a heap.
Bulk builder	`vdb_build_100m.py`	Streams BigANN over HTTP into an IVF-PQ index with checkpoints; `--offset` builds shards.
Benchmarks	`vdb_day7_bench.py` + JSON	1 shard vs N shards, recall@10, p50/p99, batch 1 and 16.
Tests	`tests/test_vdb.py`	13 tests: exactness, persistence, re-rank, sharded == single, failure handling, RAG.



each shard = one vdb_day6_shard.py process = one VectorDB directory; it holds ids where id % 4 == shard

Figure 1. The serving architecture. The client cannot tell a router from a shard.

Headline number

1,000,000 real SIFT vectors, recall@10 = 0.959 at p99 = 0.92 ms per query on one CPU thread of an Apple M1 laptop (IVF4096,PQ16, nprobe 64, exact re-rank of the top 100). The index is 32.7 MB; the same vectors as raw float32 are 495.9 MB.

2. System design: what it is and how this project does it

System design is deciding how the pieces of a software system fit together so it meets its requirements: how fast, how big, how reliable, how cheap. Code answers "does it work?". System design answers "does it still work with 1,000x the data, 100x the users, and one machine on fire?"

The questions every design answers

Question	Name	This project's answer
How fast must one request be?	Latency (p50, p99)	p99 under 100 ms at 1B vectors; measured 0.9 ms at 1M on one thread.
How many requests per second?	Throughput (QPS)	Batch search uses all cores; each shard has a gRPC thread pool.
How much data?	Scale	1M local, 100M streamed, 1B on a cloud box. PQ makes it fit in RAM.
Does it survive a crash/restart?	Durability	Atomic save (temp dir + rename), checkpoints every 10M vectors.
What if a machine dies?	Availability / fault tolerance	Router answers from the healthy shards (partial results) or fails fast with a timeout.
How exact must answers be?	Accuracy (recall)	recall@10 at least 0.95, bought back with exact re-ranking.
What does it cost?	Cost	16 B/vector instead of 512 B: one rented box instead of a rack.
Can it grow?	Scalability	Add shards; the router and clients do not change.

The core idea: trade-offs, not perfect answers

Every choice in this project buys something by paying with something else. IVF buys speed by risking a missed neighbour. PQ buys memory by losing precision. Sharding buys capacity by adding network hops and tail latency. System design is choosing the trade you can afford, then **measuring** that you got it.

Building blocks used here, and where they appear everywhere else

Building block	In this project	Same idea in industry
Index	FAISS IVF-PQ / HNSW	B-trees in Postgres, inverted indexes in Elasticsearch
Partitioning (sharding)	id % N across shard processes	Cassandra, DynamoDB, Kafka partitions, Elasticsearch shards
Scatter-gather	Router fans out, merges top-k	Google web search, Elasticsearch query phase
RPC	gRPC + Protobuf	Google internal (Stubby), Netflix, Uber, Kubernetes, etcd
Caching / precomputation	Embed once at ingest, PQ lookup tables	CDNs, Redis, materialised views
Compression	Product quantization	Parquet columnar encoding, video codecs
Checkpointing	Build resumes every 10M vectors	Spark, ML training checkpoints, database WALs

Building block	In this project	Same idea in industry
Stateless front + stateful back	Router holds no data; shards do	Web servers in front of databases

How it makes production faster

Separating the router (stateless) from shards (stateful) lets you scale each independently: more routers for more QPS, more shards for more data. Parallel fan-out means a query over 4 shards costs roughly the time of the slowest shard, not the sum of all four.

3. Embeddings and vectors

What it is

An **embedding** is a list of numbers produced by a neural network such that things with similar meaning get nearby lists. "neurons that fire together wire together" and "learning strengthens synapses" share few words but land close together. Gemini's `gemini-embedding-001` turns text into 3,072 numbers. BigANN's SIFT vectors are 128 numbers describing small image patches.

In this project

- `day2_embeddings.embed()` sends a whole list of texts in **one batched API call**.
- `vdb_day5_rag.py` uses task types: `RETRIEVAL_DOCUMENT` for facts, `RETRIEVAL_QUERY` for questions. The model embeds the two sides slightly differently so questions match answers, not other questions.
- Every RAG vector is **L2-normalised** (length 1). Then cosine similarity equals the dot product, so the DB can use the fast inner-product metric.
- Embedding happens once at ingest and is stored. Each question costs one embedding call, not 17.

Why

Keyword search fails on paraphrases. Embeddings capture meaning, which is what RAG, recommendation and visual search need.

Where it is used in the market
Semantic search (Google, Bing), RAG chatbots, recommendation (Spotify, YouTube, Netflix item and user embeddings), **fashion visual search** ("shop the look", similar items in e-commerce apps, Pinterest Lens), duplicate detection, fraud detection, face recognition.

How it makes production faster
Batching: 100 texts in one call instead of 100 calls cuts network round trips 100x. Embedding at ingest and persisting the vectors means queries never re-embed the corpus.

What it costs you
3,072 floats = 12 KB per text. A billion of those is 12 TB, which is why compression (PQ, section 9) exists.

4. Distance metrics: L2 and inner product

L2 (Euclidean) distance: the straight-line distance; smaller = closer. Used for BigANN/SIFT, which is what that benchmark is defined on. **Inner product (dot product)**: bigger = more similar. On unit vectors it equals **cosine similarity**, the standard for text embeddings.

```
VectorDB(dim, factory, metric="l2")   # BigANN benchmark: score = squared distance, ascend
ing
VectorDB(dim, "Flat",  metric="ip")   # RAG: vectors normalised, score = cosine, descendin
g
```

The metric matters beyond scoring: the router's heap merge must know whether smaller or bigger wins (`merge_topk` flips the sign for `ip`), and the re-ranker recomputes the exact score in the same metric.

The similarity floor

Cosine between unrelated texts is never 0 with Gemini; junk scores about 0.52 to 0.62 on your corpus. `vdb_day5_rag.py calibrate` embeds 9 junk probes, takes the highest score junk reaches, adds 0.05, and stores that **floor (0.67)** inside the database's `config.json`. A floor belongs to one corpus and one model, so it lives with the data, not in the code.

> **How it makes production faster**
> Questions with nothing above the floor skip the LLM call entirely: no cost, no hallucinated answer, lower latency.

5. Exact search, ground truth, and recall@k

Brute force (Flat)

Compare the query with every vector and keep the best k. Always correct; cost grows linearly with data. At 1M x 128 dims that is 128M multiply-adds per query. FAISS calls it `IndexFlat`; `faiss.knn()` does it in one call.

Ground truth

Run brute force once, save the true top-100 per query (`indexes/bigann_gt_*.npy`). Every fast index is graded against it. The 100M builder computes ground truth **during** the streaming pass: exact top-100 against each 500k slice, merged into a running top-100 (`merge_gt`). One pass gives the index and its answer key.

recall@10

For each query: how many of the true 10 nearest did the index return? Average over all queries. 0.95 means on average 9.5 of the 10 true neighbours were found. It is the accuracy number of approximate search.

```
def recall_at_k(I, GT, k=10):
    return sum(len(set(a[:k]) & set(b[:k])) for a, b in zip(I, GT)) / (len(GT) * k)
```

Where it is used in the market
Every ANN benchmark (ann-benchmarks.com, big-ann-benchmarks, VectorDBBench) reports recall@k vs QPS/latency. Vendors tune to a recall target, usually 0.9 to 0.99.

What it costs you
Flat is the right choice below ~100k vectors (your RAG corpus uses it). Above that, approximate indexes win by orders of magnitude.

6. Approximate nearest neighbour (ANN) and LSH

ANN gives up guaranteed exactness for speed: look at a small, smart subset of the data instead of all of it. Three families: **hashing** (LSH), **graphs** (HNSW), **partitioning + compression** (IVF, PQ).

LSH (your day5_lsh.py)

Random hyperplanes slice the space; each vector gets a bit for which side of each plane it lies on. Similar vectors tend to share bit patterns, so you only compare within the same bucket. Simple and fast to build, but needs many tables for good recall, so it lost to graphs and IVF in practice.

> **Where it is used in the market**
> LSH survives in near-duplicate detection (MinHash for web pages, Spark's LSH), not as the main index of modern vector databases.

7. HNSW: the graph index

What it is

Hierarchical Navigable Small World: every vector is a node linked to about M near neighbours. A few nodes are promoted to sparse upper layers like an express train. Search starts at the top, greedily walks toward the query, drops a layer, repeats. Each step only looks at a handful of neighbours.

Knob	When it is paid	Effect
M (edges per node)	Build time + memory, forever	More edges = better recall, ~M x 8 bytes extra per vector
efConstruction	Build time	Bigger = better graph, slower build
efSearch	Every query; changeable live	Bigger = more recall, more ms. Tune this first.

In this project

Plan days 1–2 (`day6_hnsw.py`, `vdb_day2_hnsw_1m.py`) swept M x efSearch at 1M vectors and plotted the recall–latency curve. The `VectorDB` class accepts `factory="HNSW32"` and sets `efSearch` per query (`test_hnsw_params` proves it).

Why not HNSW for 1B?

HNSW stores full vectors plus the graph: about 770 bytes per 128-d vector at M=32. At 1B that is ~770 GB of RAM. IVF-PQ needs ~24 GB. HNSW is the best choice when the data fits in RAM; IVF-PQ when it does not.

Where it is used in the market
The default index of pgvector, Qdrant, Weaviate, Redis, Elasticsearch/OpenSearch (via Lucene), MongoDB Atlas Vector Search and many more. Milvus offers it next to IVF variants.

How it makes production faster
Search cost grows roughly with log(N), so 1M to 100M vectors adds only a few hops. Typical sub-millisecond to a few ms per query at high recall.

What it costs you
Memory-hungry, slow to build, deletes are awkward (nodes are tied into the graph).

8. IVF: the inverted file (partitioning)

What it is

Run k-means on the data to get `nlist` centroids (4,096 at 1M). Each vector goes into the list of its nearest centroid. A query finds its `nprobe` nearest centroids and only scans those lists. With nprobe = 16 of 4,096 lists it touches ~0.4% of the data.

- **nlist** (build time): rule of thumb ~4 x sqrt(N). `nlist_for()` in the bench rounds it to a power of 2.
- **nprobe** (query time): the IVF equivalent of efSearch. More lists = more recall, more ms.
- **Training on a sample**: k-means on 200k vectors is enough; the remaining 800k (or 999M) are only assigned. This is how every billion-scale build works.

The name

"Inverted file" comes from search engines: an inverted index maps word -> documents. Here it maps centroid -> vectors. Same structure, same reason: skip everything not in the relevant lists.

Where it is used in the market
FAISS IVF indexes at Meta; Milvus IVF_FLAT / IVF_SQ8 / IVF_PQ; pgvector `ivfflat`; Google ScaNN (Vertex AI Vector Search) uses partitioning + quantization in the same spirit.

How it makes production faster
Cuts the vectors scanned per query by 100–1000x. Also makes sharding natural: lists are independent.

What it costs you
A true neighbour sitting just across a cell border is missed unless nprobe is large enough; recall plateaus. Re-ranking does not fix a neighbour that was never a candidate.

9. PQ: product quantization (compression)



512 B -> 16 B per vector (32x). 1B vectors: 512 GB raw -> ~16 GB of codes + 8 GB of ids

Figure 2. One vector's journey into an IVF4096,PQ16 index.

What it is

Cut each 128-d vector into m = 16 pieces of 8 dimensions. For each piece position, learn 256 typical pieces (a codebook) with k-means. Store each piece as the 1-byte id of its nearest codebook entry. 128 floats (512 bytes) become 16 bytes: **32x smaller**.

Asymmetric distance computation (ADC)

At query time the query stays exact. For each of the 16 positions, precompute the distance from the query piece to all 256 codebook entries: a 16 x 256 lookup table. The distance to any stored vector is then 16 table lookups and adds, instead of 128 multiply-adds on floats. That is why PQ is faster as well as smaller.

Your measurements (100k vectors, plan day 3)

PQ m	bytes / vector	recall@10 (no IVF, no re-rank)
8	8	0.314
16	16	0.474
32	32	0.683

Compression alone loses a lot of recall. That is expected, and the next section buys it back.

Where it is used in the market
FAISS (Meta), Milvus IVF_PQ, ScaNN's anisotropic quantization, Pinecone and other managed services use quantization internally; Elasticsearch, Qdrant and Weaviate offer scalar/binary/product quantization to cut RAM.

How it makes production faster
4x to 32x less RAM means the index fits in memory on one box, so no disk reads per query. Smaller data also means more of it fits in CPU cache, and memory bandwidth, the real bottleneck, drops.

What it costs you
Approximate distances: ranking errors among close candidates. Must be trained before adding data.

10. IVF-PQ + exact re-ranking

Combine both: IVF chooses which lists to scan, PQ makes scanning them cheap. FAISS builds it from one string: `index_factory(128, "IVF4096,PQ16")`. Then fix PQ's ranking errors:

- Ask the index for the top **100** candidates (cheap, approximate).
- Fetch those 100 **exact** float vectors from the raw store (`raw.faiss`, an IndexIDMap2 over IndexFlat).
- Recompute exact distances, keep the best 10 (`VectorDB._rerank`).

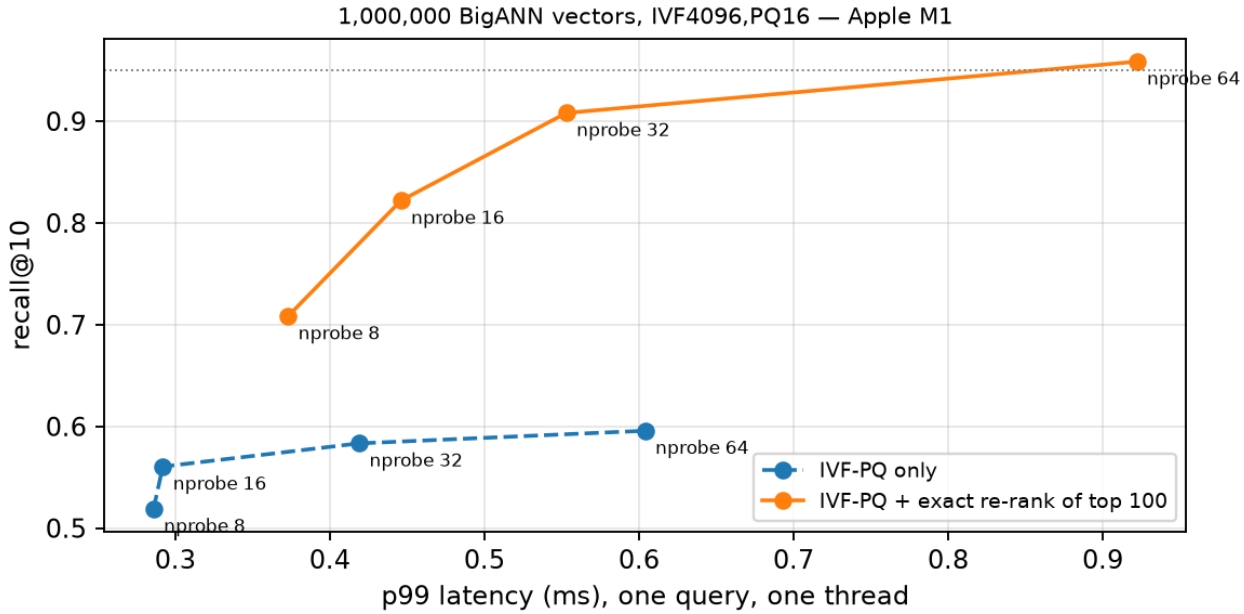


Figure 3. Measured: re-ranking moves recall from ~0.6 to ~0.96 for well under 1 ms.

nprobe	re-rank	recall@10	p50 ms	p99 ms
8	off	0.5189	0.197	0.286
8	100	0.7083	0.285	0.373
16	off	0.5605	0.219	0.292
16	100	0.8219	0.325	0.446
32	off	0.5833	0.292	0.419
32	100	0.9081	0.407	0.553
64	off	0.5957	0.438	0.604
64	100	0.9586	0.630	0.922

Read the table like an engineer: without re-ranking, recall stalls near 0.6 no matter how many lists you scan, because the limit is PQ precision, not coverage. With re-ranking, nprobe becomes the dial again.

> **Where it is used in the market**
> The standard production pattern: "retrieve then re-rank". Search engines retrieve with a cheap index and re-rank with an expensive model; vector DBs retrieve with quantized codes and re-score with full vectors (often kept on SSD). RAG pipelines add a third stage, a cross-encoder re-ranker.

How it makes production faster
You pay exact cost on 100 vectors instead of 1,000,000. The expensive step runs on 0.01% of the data.

What it costs you
The raw vectors must be stored somewhere: 512 MB per 1M here (RAM), or on NVMe at billion scale with ~100 random reads per query.

11. FAISS: the engine

FAISS (Facebook AI Similarity Search) is Meta's open-source C++ library with Python bindings. It provides the index algorithms; you built the database around them.

FAISS feature	Used for
`index_factory("IVF4096,PQ16")`	Build any index from a string; swap designs without code changes.
`IndexIDMap2`	Your own 64-bit ids (global across shards) + `reconstruct_batch` by id for re-ranking.
`write_index` / `read_index`	Persistence; `IO_FLAG_MMAP` maps the file instead of loading it.
`faiss.knn`	Exact brute-force search for ground truth.
`omp_set_num_threads`	1 thread for latency tests (like one live request), all cores for builds and batches.
`try_extract_index_ivf`	Reach inside wrappers to set nprobe.

Decision log: on 19 Sep you cut the Rust rewrite and from-scratch HNSW/PQ. Re-implementing a library that Meta tuned for years is learning, not product. The product is what sits around the index: persistence, serving, sharding, RAG, honest benchmarks.

Where it is used in the market
FAISS is inside or behind many vector products and research systems; Milvus embeds it (Knowhere); it is the reference implementation in most ANN research papers.

How it makes production faster
SIMD (NEON on your M1, AVX on cloud x86), cache-aware layouts, OpenMP parallelism, optional GPU. Python only orchestrates; the hot loop is compiled C++.

12. Benchmarking like a database engineer

p50 and p99, never the mean

p50 (median): the typical request. **p99**: the slowest 1 in 100. Users feel p99, and at scale "1 in 100" is thousands of people per minute. Means hide tails: 99 requests at 1 ms + 1 at 500 ms average 6 ms, which describes nobody. The project goal is stated as p99.

The rules used in every script

- **Warm up** first (20–50 queries untimed): first calls pay for page faults and cold caches.
- **Many queries** (1,000): percentiles need samples.
- **One query at a time** with one thread for latency; batched, all threads for throughput (QPS).
- **Log the hardware** with every result: `hardware_info()` writes chip, cores, RAM, Python and FAISS version into each JSON.
- **Recall and latency together**: a latency number without its recall is meaningless (any index is fast at recall 0.1).
- **Pareto frontier / operating point**: the cheapest setting that clears the recall target. Here: nprobe 64 + re-rank 100.

Where it is used in the market
Every serious vendor benchmark (VectorDBBench, ann-benchmarks, big-ann-benchmarks NeurIPS'21/'23) reports recall-vs-throughput curves with hardware. SRE teams set SLOs as p99 (or p99.9) latency.

How it makes production faster
You cannot make production faster without knowing where time goes. Percentiles find tail problems that averages hide; per-shard `server_ms` in the gRPC response shows whether time is spent searching or on the network.

13. The database layer (VectorDB class)

A FAISS index answers "which row numbers are closest?". A database must answer five more questions. This section is about `vdb_day5_db.py`.

13.1 Ids: IndexIDMap2

You choose ids (`db.add(X, ids=...)`) or they auto-increment and continue after a reload. Ids are 64-bit and **global**: shard 2 stores ids 2, 6, 10, ... so results from different shards never collide and need no translation. `IndexIDMap2` also keeps a reverse map so `reconstruct_batch(ids)` fetches vectors by id.

13.2 Metadata: SQLite

Each id maps to a JSON document (`{"text": ..., "source": ...}`) in a SQLite table. Search returns `Hit(id, score, meta)`. Metadata for all hits in a batch is fetched with one `IN (...)` query, chunked under SQLite's 999-parameter limit.

> **Where it is used in the market**
> Every vector DB pairs vectors with a payload store: Pinecone metadata, Qdrant payloads, Weaviate objects, pgvector rows next to normal columns. SQLite is the most deployed database in the world (every phone, every browser).

13.3 Persistence and atomic save

```
<dir>/config.json   dim, metric, factory, search defaults, next_id, extra (e.g. RAG floor)
<dir>/index.faiss   the ANN index
<dir>/raw.faiss     exact vectors for re-rank (optional)
<dir>/meta.sqlite   id -> JSON
```

`save()` writes everything into `<dir>.tmp`, then renames directories. A rename is atomic on one file system: after a crash you have either the old database or the new one, never half of each. The SQLite copy uses the online backup API so it is consistent even while in use.

> **Where it is used in the market**
> The same write-temp-then-rename trick is used by git, package managers, editors and database checkpoints. Full databases go further with a write-ahead log (WAL), cut from this project's scope.

13.4 mmap

`VectorDB.load(path, mmap=True)` maps the index file into memory instead of reading it. Load is instant and the OS pages in only the parts queries touch. On the cloud box this lets a 24 GB index serve before it is fully in RAM, and lets several processes share one copy of the file in the page cache.

> **What it costs you**
> The first queries hit disk (slower p99 until warm). On NVMe that is tens of microseconds per page; on a network disk, much more.

13.5 Concurrency

FAISS searches are read-only and run in parallel from many gRPC threads. Writes (`add`, `train`, `save`) take a lock, and SQLite access has its own lock. This is the simplest correct policy: many readers, one writer at a time.

> **How it makes production faster**
> Reads never wait for each other, so a shard serves as many concurrent searches as it has cores.

14. RAG: retrieval-augmented generation

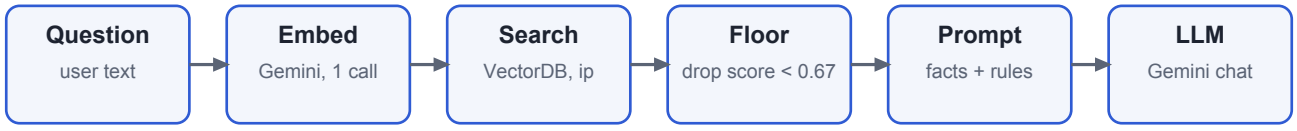


Figure 4. The RAG pipeline in vdb_day5_rag.py.

What it is

An LLM only knows what it was trained on. RAG retrieves relevant facts from your own data and puts them in the prompt, with rules: answer **only** from these facts, say "I don't know" otherwise. The model becomes a reader of your database instead of a guesser.

What the DB port changed (day3 -> day5)

	day3_rag.py	vdb_day5_rag.py
Storage	Python list, rebuilt each run	VectorDB directory on disk
Embedding calls per question	17+ (corpus re-embedded) + 9 probes	1
Floor	recomputed on import	measured once, saved in config.json
Adding facts	edit code	`ingest file.txt` (dedupes, batches of 100)
Sampling	fixed	`--temperature`, `--top-p`

Temperature and top-p

Temperature sharpens or flattens the next-word probabilities: 0 = always the most likely word (repeatable, good for factual RAG), higher = more varied. **Top-p** keeps only the smallest set of words whose probabilities add up to p. For grounded answers: low temperature, top-p around 0.9.

Where it is used in the market
Customer-support bots, internal knowledge assistants (Notion AI, Glean), coding assistants over your repo, legal/medical search, e-commerce assistants ("which of these jackets is waterproof?").

How it makes production faster
Embedding once at ingest removes the slowest repeated step. The floor turns unanswerable questions into an instant "I don't know" with zero LLM cost. A small, relevant context is also a cheaper, faster prompt than dumping everything.

15. Protobuf and gRPC

Protobuf

Protocol Buffers is a schema language plus a compact binary encoding. `vecdb.proto` defines the messages; `grpc_tools.protoc` generates `vecdb_pb2.py` (messages) and `vecdb_pb2_grpc.py` (client stub + server base). Both sides share one contract, so a field cannot be misspelled on one side.

Vectors as bytes

Queries travel as `bytes` produced by `numpy.tobytes()`, not as `repeated float`. 1,000 x 128 floats as bytes is one memory copy; as a repeated field it is 128,000 separate encodes and decodes. The receiver rebuilds the array with `np.frombuffer` without copying.

gRPC

A remote procedure call framework on HTTP/2: you call `stub.Search(request)` as if it were a local function. Used here for:

- **Futures**: `stub.Search.future(...)` starts all shard calls at once; the router then collects them.
- **Deadlines**: every call has a timeout, so one hung shard cannot hang the router.
- **Status codes**: `INVALID_ARGUMENT` for a wrong dimension, `UNAVAILABLE` when all shards fail.
- **Trailing metadata**: the router reports partial results (`partial: shard3 UNAVAILABLE`).
- **Message size**: raised to 256 MB for bulk inserts (default is 4 MB).

	REST + JSON	gRPC + Protobuf
Encoding	text; floats printed as strings	binary; floats as raw bytes
Contract	docs / OpenAPI, optional	.proto, required, code-generated
Connection	HTTP/1.1, often one request per connection	HTTP/2, many streams multiplexed on one connection
Best for	public APIs, browsers	service-to-service, high QPS, low latency

Where it is used in the market
Google (gRPC is the open version of its internal Stubby), Netflix, Uber, Square, Kubernetes and etcd, Milvus, Qdrant and Weaviate client APIs. Most microservice backends talk gRPC internally.

How it makes production faster
Binary encoding and persistent multiplexed connections remove JSON parsing and TCP/TLS handshakes from every request: typically several times less CPU and latency per call than JSON over HTTP/1.1.

16. Sharding, scatter-gather and the heap merge

Sharding

Split the data across N independent servers. Each vector lives on shard `id % N`. Each shard has 1/N of the data, so it searches 1/N of the vectors, needs 1/N of the RAM, and can live on a different machine.

Scatter-gather

The router sends the same query to every shard in parallel (scatter), waits, and combines the answers (gather). Every shard must be asked: any of them could hold the true nearest neighbour.

k-way merge with a heap

Each shard returns its own best k, already sorted. The global best k are somewhere in those N sorted lists. `heapq.merge` keeps a heap of the N current heads and pops the smallest k times: O(k log N) work, without sorting everything. With exact (re-ranked) scores the merged answer is **identical** to one big database, which `test_sharded_equals_single` proves for both metrics.

```
streams = [_stream(S[q], I[q], p, sign) for p, (S, I, _) in enumerate(parts)]
for r, (s, i, p, j) in enumerate(itertools.islice(heapq.merge(*streams), k)):
    S_out[q, r], I_out[q, r] = sign * s, i
```

> **A real bug we hit**
> The first version built each shard's stream as an inline generator expression inside the loop. Python looks up loop variables when the generator runs, not when it is created, so all four streams read the last shard's arrays. The merge returned duplicates. The sharded-equals-single test caught it; the fix is a helper function (`_stream`) that binds the arrays at call time.

Tail latency amplification

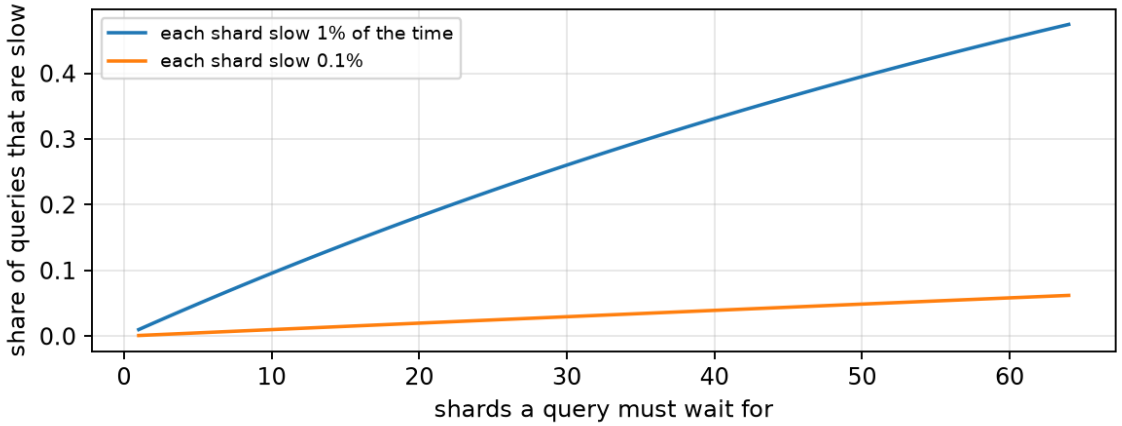


Figure 5. A query waits for its slowest shard: 1 - (1 - p)^N of queries are slow.

If one shard is slow 1% of the time, a query that waits for 4 shards is slow 3.9% of the time, and with 64 shards 47%. So sharding lowers per-shard work but raises the chance of hitting a slow one. Fixes used in industry: timeouts with partial answers (implemented: `allow_partial`), hedged requests (send a duplicate to a replica after p95), replicas, and keeping shards few and fat.

Measured: 1 shard vs N shards, end to end through gRPC

config	nprobe	re-rank	recall@10	p50 ms	p99 ms
1 shard	8	off	0.5189	1.589	4.248
1 shard	8	100	0.7083	2.469	6.091
1 shard	16	off	0.5605	1.683	3.110
1 shard	16	100	0.8219	2.299	5.371
1 shard	32	off	0.5833	1.153	3.115
1 shard	32	100	0.9081	1.303	3.011
1 shard	64	off	0.5957	1.235	3.243
1 shard	64	100	0.9586	1.410	2.708
4 shards	8	off	0.5405	2.855	9.130
4 shards	8	100	0.7885	3.666	14.551
4 shards	16	off	0.5720	2.236	3.949
4 shards	16	100	0.8894	3.253	7.961
4 shards	32	off	0.5880	2.759	7.119
4 shards	32	100	0.9538	3.602	26.021
4 shards	64	off	0.5936	2.959	5.673
4 shards	64	100	0.9850	3.173	5.600

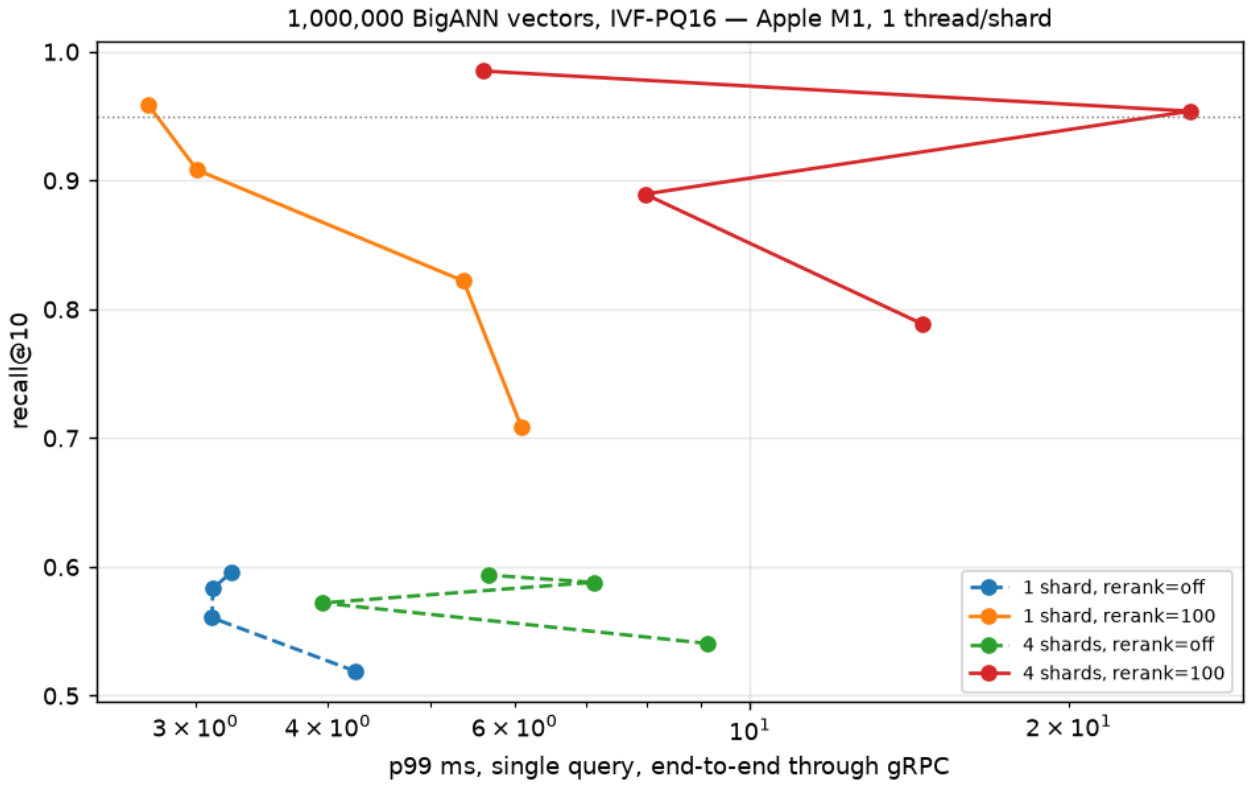


Figure 6. Recall vs p99 through the router, 1 shard vs N shards.

What the numbers say. (1) gRPC + Python adds about 1–2 ms per query: the same 1-shard index measured 0.92 ms p99 in-process (section 10) and 2.7 ms through the router. (2) At the same nprobe, 4 shards reach **higher recall** (0.985 vs 0.959 at nprobe 64), because each shard scans 64 of its own 2,048 lists, 4x more data in total. (3) 4 shards are **slower and spikier** here: p99 jumps to 8–26 ms at

some settings. That is tail amplification plus contention: four shard processes, the router and the benchmark share 8 cores (4 fast + 4 efficiency on the M1), so one shard landing on a slow core or being descheduled delays the whole query.

The lesson is the real one engineers learn: **sharding is for data that does not fit one machine, not a free speedup.** On one laptop, 1M vectors are served best by one shard. At 1B, where one shard cannot hold the data, each shard on its own cores is the only option, and the tail fixes below decide the p99.

Where it is used in the market
Elasticsearch/OpenSearch shards, Milvus query nodes and segments, Pinecone pods, Vespa content nodes, Google search's index serving, Cassandra and DynamoDB partitions.

How it makes production faster
Capacity grows by adding machines instead of buying a bigger one. Each shard searches less data, so per-shard latency drops, and shards work in parallel, so a 1B search costs about one 250M search plus a merge.

What it costs you
Network hops, merge cost, tail amplification, and more machines to operate. Shard only when one box cannot hold or serve the data.

17. Building at scale: streaming, sampling, checkpointing

`vdb_build_100m.py` builds an index over data bigger than the disk and RAM of the machine.

Technique	How	Why
HTTP range requests	`Range: bytes=start-end` pulls 64 MB slices of the 128 GB BigANN file	Never store the raw dataset; 100M vectors flow through 8 GB of RAM
Prefetching	4 slices downloading in a thread pool while the current one is encoded	Network and CPU work overlap instead of taking turns
Train on a sample	IVF centroids + PQ codebooks learned from the first 1M vectors	k-means on 1B points is unnecessary and slow; the sample has the same distribution
Ground truth in the same pass	exact top-100 per slice, merged into a running top-100	One pass yields the index and the answer key
Checkpoints	every 10M vectors: write index + GT + progress JSON (temp + rename)	A dropped connection or a sleeping laptop resumes instead of restarting
`--offset` shards	build vectors [offset, offset+n) with global ids via `add_with_ids`	Each 1B shard builds independently, in parallel, and its ids already match the router

Where it is used in the market
Spark/Flink streaming jobs, ML training checkpoints, data-lake ingestion. Every billion-scale ANN index (FAISS, DiskANN, ScaNN) trains on a sample and encodes the rest.

How it makes production faster
Overlapping I/O and compute, never reprocessing completed work, and building shards in parallel turn a multi-day build into hours.

18. The 1B run: cloud, memory math and cost

Memory math

What	Per vector	1B vectors
Raw float32, 128 dims	512 B	512 GB
Raw uint8 (BigANN native)	128 B	128 GB
HNSW32 (vectors + graph)	~770 B	~770 GB
IVF-PQ16 codes	16 B	16 GB
+ 64-bit ids in inverted lists	8 B	8 GB
IVF-PQ16 total	~24 B	~24 GB + centroids (tiny)

That is why IVF-PQ is the only option that fits 1B vectors on one rentable box with RAM to spare.

Runbook (the plan; not yet run)

```
# on a spot/preemptible box with >= 64 GB RAM and NVMe (4 shards x 250M):
for s in 0 1 2 3; do
  python vdb_build_100m.py --n 250000000 --offset $((s*250000000)) \
         --nlist 65536 --tag 1B_s$s &
done; wait
for s in 0 1 2 3; do
  python vdb_day6_shard.py --index indexes/bigann1B_s${s}_ivfpq16.index \
         --port $((50051+s)) --nprobe 64 &
done
python vdb_day7_bench.py --connect localhost:50051,localhost:50052,localhost:50053,localho
st:50054 \
       --gt indexes/bigann1B_s0_gt.npz,indexes/bigann1B_s1_gt.npz,indexes/bigann1B_s2_gt.n
pz,indexes/bigann1B_s3_gt.npz \
       --tag 1B
```

Spot instances

Spare cloud capacity sold at a large discount that can be taken back with short notice. Fine for a checkpointed batch job; the build resumes from the last 10M checkpoint. Tear the instance down the moment results are saved; idle machines are the main source of surprise bills.

> **Where it is used in the market**
> Batch ML training, rendering, CI, and index builds commonly run on spot capacity with checkpointing.

19. Testing: proving it works

Test	What it proves
flat_is_exact_and_metadata_roundtrips	Flat search = brute force; metadata follows ids
custom_ids_and_auto_ids_continue	Your own ids work; auto ids never collide with them
ip_metric	Inner product returns the true neighbours, scores descending
save_load_roundtrip	Index, raw vectors, metadata, config, floor and next id all survive a restart; overwrite is safe
mmap_load	Memory-mapped load returns identical results
rerank_buys_recall_back	Exact re-ranking raises recall by more than 0.2
hnsw_params	efSearch is actually applied through the wrapper
bare_index_file	Indexes from the bulk builder (with offset ids) load and search correctly
merge_topk_heap	The k-way merge is right for both metrics and skips empty slots
sharded_equals_single (l2, ip)	4 gRPC shards + router == one database, exactly; inserts route by id; router-as-server works
partial_failure_still_answers	A dead shard yields partial results, not an error
rag_with_fake_embedder	Ingest dedupes, floor persists, junk is refused, no API calls in tests

```
.venv/bin/python -m pytest tests -q      # 13 passed
```

> **How it makes production faster**
> Tests let you change the system without fear: the sharding bug in section 16 was caught in seconds, not in production. Fakes (a fake embedder and chat function) keep tests free and offline.

20. Summary: every technique and what it buys

Technique	What it buys	What it costs	Who uses it
Embeddings	Search by meaning	Model calls, 3–12 KB per item	Every semantic search / RAG / recsys
Batching	100x fewer round trips	A little latency for the batch	All ML APIs, databases
Normalise + inner product	Cosine as a fast dot product	Must normalise everywhere	Text embedding stacks
HNSW	~log N search, high recall	RAM, slow build	pgvector, Qdrant, Weaviate, Elastic
IVF	Scan ~1% of data	Border misses	FAISS, Milvus, pgvector ivfflat
PQ	32x less memory, faster scans	Approximate distances	FAISS, Milvus, ScaNN
Re-ranking	Recall back to 0.95+	Raw vectors stored	All production retrieval
SQLite metadata	Results carry meaning	Second store to keep in sync	Every vector DB payload store
Atomic save / checkpoints	Crash safety, resumable builds	Temporary 2x disk	Databases, ML training
mmap	Instant load, shared memory	Cold first queries	Lucene, LMDB, DiskANN
gRPC + Protobuf	Fast typed RPC	Less human-readable	Google, Netflix, Kubernetes
Sharding + scatter-gather	Horizontal scale	Tail latency, more machines	Elastic, Milvus, Pinecone
Heap merge	O(k log N) combine	Python loop per query	Search engines, merge sort
Timeouts + partial results	Availability	Rarely incomplete answers	Google search, Elastic
p99 benchmarking	Truth about user experience	Many queries, warmup	Every SRE team

21. Numbers and resume lines

Measured (Apple M1, 8 cores, 8 GB RAM, 1 thread, in RAM):

- 1,000,000 BigANN vectors, IVF4096,PQ16: recall@10 0.959, p50 0.63 ms, p99 0.92 ms (nprobe 64, re-rank 100).
- Index 32.7 MB vs 495.9 MB raw float32 (PQ16, 32x on the codes).
- Re-ranking: recall 0.60 -> 0.96 at nprobe 64 for +0.3 ms p99.
- Through gRPC: 1 shard recall 0.959 at p99 2.7 ms; 4 shards recall 0.985 at p99 5.6 ms (same laptop, 1 thread/shard).
- RAG: 1 embedding call per question (from 17+), calibrated floor 0.67 stored with the data.

Still to measure: the 1B cloud run (section 18). Report 1B numbers only once they exist, with the instance type next to them.

Resume paragraph (fill the brackets after the cloud run)

> **Draft**
> Built a sharded vector database in Python on FAISS: IVF-PQ indexes with exact re-ranking, id/metadata store (SQLite), atomic persistence and mmap loading, gRPC shard servers and a scatter-gather router with k-way heap merge and partial-failure handling. Streamed BigANN over HTTP into checkpointed shard builds. 1M vectors: recall@10 0.96 at 0.9 ms p99 on one laptop core; [1B vectors: recall@10 X at Y ms p99 on <instance>]. Ported a Gemini RAG pipeline onto it with a calibrated similarity floor.

22. Glossary

Term	Meaning
ANN	Approximate nearest neighbour search: fast, almost always right
Codebook	The 256 typical sub-vectors PQ learned for one piece position
Centroid	Centre of a k-means cluster; one per IVF list
efSearch	HNSW query-time breadth; more = more recall, more ms
Embedding	Vector a model produces so that similar meaning = nearby vectors
Fan-out	Sending one request to many servers
Ground truth	The exact answer from brute force, used for grading
k	How many neighbours a query asks for (10 here)
mmap	Mapping a file into memory; the OS loads pages on demand
nlist / nprobe	IVF lists built / IVF lists scanned per query
p50 / p99	Median / 99th-percentile latency
PQ	Product quantization: compress vectors into a few bytes
QPS	Queries per second (throughput)
Re-rank	Re-score a candidate list with an exact, more expensive method
Recall@k	Share of the true k neighbours that were returned
Shard	One slice of the data served by one process/machine
Spot instance	Discounted cloud machine that can be reclaimed
Tail latency	The slow end of the latency distribution (p99, p99.9)